

DATA STRUCTURES USING C

Lab 1:

```
main.c
1  #include <stdio.h>
2
3  int main() {
4      int a[10], n, i, max, min;
5
6      scanf("%d", &n);
7
8      for(i = 0; i < n; i++)
9          scanf("%d", &a[i]);
10
11     max = min = a[0];
12
13     for(i = 1; i < n; i++) {
14         if(a[i] > max) max = a[i];
15         if(a[i] < min) min = a[i];
16     }
17
18     printf("%d %d", max, min);
19     return 0;
20 }
21
22
```

input

```
5
7 8 1 2 3
8 1
```

LAB 2:

```
main.c
1  #include <stdio.h>
2
3  int fact(int n) {
4      if (n == 0)
5          return 1;
6      return n * fact(n - 1);
7  }
8
9  int main() {
10     int n;
11     scanf("%d", &n);
12     printf("%d", fact(n));
13     return 0;
14 }
15
```

input

```
5
120
...Program finished with exit code 0
Press ENTER to exit console
```

LAB 3:

```
main.c
1  #include <stdio.h>
2
3  int fib(int n) {
4      if(n <= 1)
5          return n;
6      return fib(n - 1) + fib(n - 2);
7  }
8
9  int main() {
10     int n;
11     scanf("%d", &n);
12     printf("%d", fib(n));
13     return 0;
14 }
15
16
```

input

10
55

...Program finished with exit code 0
Press ENTER to exit console.

LAB 4: Singly linked list

Insert at beginning:

```
main.c
1  #include<stdio.h>
2  #include<stdlib.h>
3  struct Node{
4      int data;
5      struct Node *next;
6  };
7  struct Node *head=NULL;
8  void insertBeginning(int value){
9      struct Node *newNode=(struct Node*)malloc(sizeof(struct Node
10 ));
11     newNode->data=value;
12     newNode->next=head;
13     head=newNode;
14 }
15 void display(){
16     struct Node *temp=head;
17     while(temp!=NULL){
18         printf("%d -> ",temp->data);
19         temp=temp->next;
20     }
21     printf("NULL\n");
22 }
23 int main(){
24     insertBeginning(10);
25     insertBeginning(20);
26     insertBeginning(30);
27     display();
28 }
```

Output

30 -> 20 -> 10 -> NULL

Insert at End:

main.c

```
1  #include<stdio.h>
2  #include<stdlib.h>
3  struct Node{
4      int data;
5      struct Node *next;
6  };
7  struct Node *head=NULL;
8  void insertEnd(int value){
9      struct Node *newNode=(struct Node*)malloc(sizeof(struct Node
10     ));
11     newNode->data=value;
12     newNode->next=NULL;
13     if(head==NULL){
14         head=newNode;
15         return;
16     }
17     struct Node *temp=head;
18     while(temp->next!=NULL)
19         temp=temp->next;
20     temp->next=newNode;
21 }
22 void display(){
23     struct Node *temp=head;
24     while(temp!=NULL){
25         printf("%d -> ",temp->data);
26         temp=temp->next;
27     }
28     printf("NULL\n");
29 }
```

```
29 int main(){
30     insertEnd(10);
31     insertEnd(20);
32     insertEnd(30);
33     display();
34 }
```

Output

10 -> 20 -> 30 -> NULL

Delete Node:

```

main.c
1  #include<stdio.h>
2  #include<stdlib.h>
3  struct Node{
4      int data;
5      struct Node *next;
6  };
7  struct Node *head=NULL;
8  void insertEnd(int value){
9      struct Node *newNode=(struct Node*)malloc(sizeof(struct Node
10     ));
11     newNode->data=value;
12     newNode->next=NULL;
13     if(head==NULL){
14         head=newNode;
15         return;
16     }
17     struct Node *temp=head;
18     while(temp->next!=NULL)
19         temp=temp->next;
20     temp->next=newNode;
21 }
22 void deleteBeginning(){
23     struct Node *temp=head;
24     head=head->next;
25     free(temp);
26 }
27 void display(){
28     struct Node *temp=head;
29     while(temp!=NULL){
30         printf("%d -> ",temp->data);
31         temp=temp->next;
32     }
33     printf("NULL\n");
34 }
35 int main(){
36     insertEnd(10);
37     insertEnd(20);
38     insertEnd(30);
39     deleteBeginning();
40     display();
41 }

```

Output

```
20 -> 30 -> NULL
```

LAB 5:

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  struct Node {
4      int data;
5      struct Node* next;
6  };
7  struct Node* top = NULL;
8  void push(int x) {
9      struct Node* new = malloc(sizeof(struct Node));
10     new->data = x;
11     new->next = top;
12     top = new;
13 }
14 int pop() {
15     if (top == NULL) return -1;
16     int val = top->data;
17     struct Node* temp = top;
18     top = top->next;
19     free(temp);
20     return val;
21 }

```

```

22 int peek() {
23     if (top == NULL) return -1;
24     return top->data;
25 }
26 int main() {
27     push(10);
28     push(20);
29     push(30);
30     printf("Top: %d\n", peek());
31     printf("Popped: %d\n", pop());
32     printf("Top after pop: %d\n", peek());
33     return 0;
34 }

```

```

Top: 30
Popped: 30
Top after pop: 20

```

LAB 6:

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  struct Node {
4      int data;
5      struct Node* next;
6  };
7  struct Node *front = NULL, *rear = NULL;
8  void enqueue(int x) {
9      struct Node* new = malloc(sizeof(struct Node));
10     new->data = x;
11     new->next = NULL;
12     if (rear == NULL) {
13         front = rear = new;
14         return;
15     }
16     rear->next = new;
17     rear = new;
18 }
19 int getFront() {
20     if (front == NULL) return -1;
21     return front->data;
22 }

```

```

23 int main() {
24     enqueue(10);
25     enqueue(20);
26     enqueue(30);
27     printf("Front: %d\n", getFront());
28     return 0;
29 }

```

Front: 10

LAB 7: Doubly Linked list

Insert at Beginning:

```

main.c
1  #include<stdio.h>
2  #include<stdlib.h>
3  struct Node{
4      int data;
5      struct Node *next;
6      struct Node *prev;
7  };
8  struct Node *head=NULL;
9  void insertBeginning(int value){
10     struct Node *newNode=(struct Node*)malloc(sizeof(struct Node));
11     newNode->data=value;
12     newNode->prev=NULL;
13     newNode->next=head;
14     if(head!=NULL)
15         head->prev=newNode;
16     head=newNode;
17 }
18 void display(){
19     struct Node *temp=head;
20     while(temp!=NULL){
21         printf("%d <-> ",temp->data);
22         temp=temp->next;
23     }
24     printf("NULL\n");
25 }
26 int main(){
27     insertBeginning(10);
28     insertBeginning(20);
29     insertBeginning(30);
30     display();

```

Output

10 -> 30 -> NULL

Insert At End:

```
main.c
1  #include<stdio.h>
2  #include<stdlib.h>
3  struct Node{
4      int data;
5      struct Node *next;
6      struct Node *prev;
7  };
8  struct Node *head=NULL;
9  void insertEnd(int value){
10     struct Node *newNode=(struct Node*)malloc(sizeof(struct Node));
11     newNode->data=value;
12     newNode->next=NULL;
13     if(head==NULL){
14         newNode->prev=NULL;
15         head=newNode;
16         return;
17     }
18     struct Node *temp=head;
19     while(temp->next!=NULL)
20         temp=temp->next;
21     temp->next=newNode;
22     newNode->prev=temp;
23 }
24 void display(){
25     struct Node *temp=head;
26     while(temp!=NULL){
27         printf("%d <-> ",temp->data);
28         temp=temp->next;
29     }
```

```
30     printf("NULL\n");
31 }
32 int main(){
33     insertEnd(10);
34     insertEnd(20);
35     insertEnd(30);
36     display();
37 }
```

Output

10 <-> 20 <-> 30 <-> NULL

Lab 8: Circular Linked List

Insert At Beginning:

```
main.c  [Icons] [Share] [Run]
1  #include<stdio.h>
2  #include<stdlib.h>
3  struct Node{
4      int data;
5      struct Node *next;
6  };
7  struct Node *head=NULL;
8  void insertBeginning(int value){
9      struct Node *newNode=(struct Node*)malloc(sizeof(struct Node));
10     newNode->data=value;
11     if(head==NULL){
12         newNode->next=newNode;
13         head=newNode;
14         return;
15     }
16     struct Node *temp=head;
17     while(temp->next!=head)
18         temp=temp->next;
19     newNode->next=head;
20     temp->next=newNode;
21     head=newNode;
22 }
23 void display(){
24     struct Node *temp=head;
25     do{
26         printf("%d -> ",temp->data);
27         temp=temp->next;
28     }while(temp!=head);
29     printf("(back to head)\n");
30 }
31 int main(){
32     insertBeginning(10);
33     insertBeginning(20);
34     insertBeginning(30);
35     display();
36 }
```

Output

30 -> 20 -> 10 -> (back to head)

LAB 9:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  struct Node {
4      int data;
5      struct Node* left;
6      struct Node* right;
7  };
8  struct Node* create(int x) {
9      struct Node* new = malloc(sizeof(struct Node));
10     new->data = x;
11     new->left = new->right = NULL;
12     return new;
13 }
14 struct Node* insert(struct Node* root, int x) {
15     if (root == NULL) return create(x);
16     if (x < root->data)
17         root->left = insert(root->left, x);
18     else
19         root->right = insert(root->right, x);
20     return root;
21 }
```

```
22 void preorder(struct Node* root) {
23     if (root) {
24         printf("%d ", root->data);
25         preorder(root->left);
26         preorder(root->right);
27     }
28 }
29 int main() {
30     struct Node* root = NULL;
31     root = insert(root, 10);
32     insert(root, 5);
33     insert(root, 15);
34     printf("Preorder: ");
35     preorder(root);
36     return 0;
37 }
```

Preorder: 10 5 15

LAB 10:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  struct Node {
4      int data;
5      struct Node* left;
6      struct Node* right;
7  };
8  struct Node* create(int x) {
9      struct Node* new = malloc(sizeof(struct Node));
10     new->data = x;
11     new->left = new->right = NULL;
12     return new;
13 }
14 struct Node* find_lca(struct Node* root, int n1, int n2) {
15     if (root == NULL) return NULL;
16     if (root->data == n1 || root->data == n2)
17         return root;
18     struct Node* left = find_lca(root->left, n1, n2);
19     struct Node* right = find_lca(root->right, n1, n2);
20     if (left && right) return root;
21     return left ? left : right;
22 }
23 int main() {
24     struct Node* root = create(10);
25     root->left = create(5);
26     root->right = create(15);
27     struct Node* lca = find_lca(root, 5, 15);
28     printf("LCA: %d", lca->data);
29     return 0;
30 }
```

LCA: 10

LAB 11:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  struct Node {
4      int data;
5      struct Node* left;
6      struct Node* right;
7  };
8  struct Node* create(int x) {
9      struct Node* new = malloc(sizeof(struct Node));
10     new->data = x;
11     new->left = new->right = NULL;
12     return new;
13 }
14 void find_grandchildren(struct Node* root, int val) {
15     if (root == NULL) return;
16     if (root->data == val) {
17         if (root->left) {
18             if (root->left->left) printf("%d ", root->left->left->data);
19             if (root->left->right) printf("%d ", root->left->right->data);
20         }
21         if (root->right) {
22             if (root->right->left) printf("%d ", root->right->left->data);
23             if (root->right->right) printf("%d ", root->right->right->data);
24         }
25     }
26     return;
27     find_grandchildren(root->left, val);
28     find_grandchildren(root->right, val);
29 }
30 int main() {
31     struct Node* root = create(10);
32     root->left = create(5);
33     root->right = create(15);
34     root->left->left = create(2);
35     root->left->right = create(7);
36     printf("Grandchildren: ");
37     find_grandchildren(root, 10);
38     return 0;
39 }
```

Output

Grandchildren: 2 7

Lab 12:

```
1  #include <stdio.h>
2  #define MAX 10
3  int adj[MAX][MAX] = {0};
4  void add_edge(int v, int w) {
5      adj[v][w] = 1;
6      adj[w][v] = 1;
7  }
8  int main() {
9      int n = 3;
10     add_edge(0, 1);
11     add_edge(1, 2);
12     printf("Adjacency Matrix:\n");
13     for (int i = 0; i < n; i++) {
14         for (int j = 0; j < n; j++) {
15             printf("%d ", adj[i][j]);
16         }
17         printf("\n");
18     }
19     return 0;
20 }
```

Output

Adjacency Matrix:

```
0 1 0
1 0 1
0 1 0
```

LAB 13:

```
1  #include <stdio.h>
2  #define MAX 10
3  int adj[MAX][MAX] = {0};
4  int visited[MAX] = {0};
5  void add_edge(int v, int w) {
6      adj[v][w] = 1;
7      adj[w][v] = 1;
8  }
9  void dfs(int v) {
10     printf("%d ", v);
11     visited[v] = 1;
12     for (int i = 0; i < MAX; i++) {
13         if (adj[v][i] && !visited[i]) {
14             dfs(i);
15         }
16     }
17 }
18 int main() {
19     add_edge(0, 1);
20     add_edge(1, 2);
21     add_edge(0, 2);
22     printf("DFS: ");
23     dfs(0);
24     return 0;
25 }
```

Output

DFS: 0 1 2

LAB 14:

```
1  main.c lude <stdio.h>
2  #define MAX 10
3  int adj[MAX][MAX] = {0};
4  int visited[MAX] = {0};
5  void add_edge(int v, int w) {
6      adj[v][w] = 1;
7      adj[w][v] = 1;
8  }
9  int dfs_cycle(int v, int parent) {
10     visited[v] = 1;
11     for (int i = 0; i < MAX; i++) {
12         if (adj[v][i]) {
13             if (!visited[i]) {
14                 if (dfs_cycle(i, v))
15                     return 1;
16             } else if (i != parent) {
17                 return 1;
18             }
19         }
20     }
21     return 0;
22 }
```

```
23 int detect_cycle(int n) {
24     for (int i = 0; i < n; i++) {
25         if (!visited[i]) {
26             if (dfs_cycle(i, -1))
27                 return 1;
28         }
29     }
30     return 0;
31 }
32 int main() {
33     int n = 3;
34     add_edge(0, 1);
35     add_edge(1, 2);
36     add_edge(2, 0);
37     if (detect_cycle(n))
38         printf("Cycle Detected\n");
39     else
40         printf("No Cycle\n");
41     return 0;
42 }
```

Output

Cycle Detected

LAB 15:

```
1  #include <stdio.h>
2  #define MAX 10
3  int network[MAX][MAX] = {0};
4  void add_friendship(int u1, int u2) {
5      network[u1][u2] = 1;
6      network[u2][u1] = 1;
7  }
8  int main() {
9      int n = 3;
10     add_friendship(0, 1);
11     add_friendship(1, 2);
12     printf("Friendship Matrix:\n");
13     for (int i = 0; i < n; i++) {
14         for (int j = 0; j < n; j++) {
15             printf("%d ", network[i][j]);
16         }
17         printf("\n");
18     }
19     return 0;
20 }
```

Output

Friendship Matrix:

0 1 0

1 0 1

0 1 0